SISTEME ELECTRONICE PROGRAMABILE

IV A

2026-2027

Cuprins

- Curs 1 - Procesoare digitale de semnal si sisteme PDS.
- Curs 2 - Procesoare DSP. Încapsulări. Alternative.
- Curs 3 - Reprezentarea numerelor. Aritmetici.
- Curs 4 - Calea de date
- Curs 5 - Arhitectura memoriei
- **Curs 6 – Adresarea datelor**
- Curs 7 – Setul de instrucțiuni
- Curs 8 – Mecanisme de control a funcționării
- Curs 9 – Funcționarea pipeline
- Curs 10 – Dispozitivele IO și perifericele
- Curs 11 – Facilități pentru depanare și gestionarea consumului
- Curs 12 – Exemple de procesoare digitale de semnal

Lucrare de verificare 1

Lucrare de verificare 2

Curs 6 - Adresarea datelor

Cuprins

- 6.1 Adresarea implicită
- 6.2 Adresarea imediată
- 6.3 Adresarea directă la registru
- 6.4 Adresarea directa la memorie
- 6.5 Adresarea indirectă cu registru
- 6.6 Adresarea indirectă cu registru cu pre- și post-incrementare/decrementare
- 6.7 Adresarea indirectă cu registru cu indexare
- 6.8 Adresarea indirectă cu registru cu aritmetică a adreselor modulo
- 6.9 Adresarea indirectă cu registru cu inversarea biților
- 6.10 Modurile de adresare scurte
- 6.11 Adresarea directă la memorie paginată

Adresarea datelor

- **Prin adresare se înțeleg mijloacele prin care sunt specificate locațiile operanzilor.**
- În acest capitol vom prezenta modurile de adresare disponibile la procesoarele DSP, punând accentul pe modurile cele mai folosite în aplicații, existând mai multe tipuri de adresare la DSP-uri (denumite *moduri de adresare*).
- Operanzii pot fi obținuti direct dintr-un registru sau de la o adresă de memorie internă sau externă.
- Modul de adresare a datelor este indicat, fie explicit în codul instrucțiunii, fie acesta este implicit. Este des întâlnit cazul în care o instrucțiune combină două sau mai multe moduri de adresare, folosind moduri diferite pentru operanzi diferiți.
- Majoritatea procesoarelor DSP au una sau mai multe unități de generare a adresei (AGU – Address Generating Units) care sunt dedicate calculului adreselor.

6.1 Adresarea implicită

Adresele operanzilor sunt indicate în mod implicit de către instrucțiune.

Nu se poate schimba localizarea operanzilor, care în majoritatea cazurilor se află în registre.

La procesoarele AT&T, toate instrucțiunile de înmulțire folosesc operanzii din registrele de intrare ale multiplicatorului (X și Y) și depun rezultatul în registrul produs (P). Faptul că programatorul a folosit o instrucțiune de înmulțire indică în mod implicit localizarea operanzilor. Deși sintaxa folosită de limbajul de asamblare al AT&T pentru operația de înmulțire este de forma “ P = X * Y “, aceasta ar fi putut fi scrisă mai simplu drept “ * ”, deoarece programatorul nu poate specifica alte registre în care se află operanzii.

6.2 Adresarea imediată

Operandul este inclus în cuvântul instrucțiune sau într-un cuvânt separat.

De exemplu, în limbajul de asamblare de la Analog Devices, adresarea imediată poate fi utilizată pentru încărcarea unei constante într-un registru, cum ar fi

AX0 = 1234

care specifică încărcarea constantei 1234 în registrul AX0.

- în acest exemplu valoarea constantă este specificată prin adresare imediată, în timp ce registrul destinație, AX0, este specificat folosind adresarea directă la registru.

Prin intermediul adresării imediate, cuvintele de date de lungime mică se pot coda în cuvântul instrucțiune, rezultând instrucțiuni codate cu un singur cuvânt.

Datele care au format mai mare trebuie plasate în codul instrucțiunii într-un cuvânt separat. În acest caz procesorul trebuie să citească două cuvinte din memoria de program înainte ca instrucțiunea să poată fi executată, încetinind execuția programului.

6.3 Adresarea directă la registru

Datele ce trebuie adresate se află într-un registru. Programatorul specifică registrul ca parte a instrucțiunii.

De exemplu, folosind limbajul de asamblare de la Texas Instruments, instrucțiunea

SUBF R1, R2

scade valoarea aflată în registrul R1 din cea aflată în registrul R2 și plasează rezultatul în registrul R2.

Diferența dintre adresarea directă la registru și cea implicită constă în posibilitatea programatorului de a specifica registrele utilizate, în timp ce în cazul adresării implicite acestea sunt specificate de instrucțiune și nu pot fi schimbate.

De exemplu, instrucțiunea de scădere a procesorului de mai sus ar fi putut folosi oricare două registre din cele opt registe existente pe cip la acest procesor. Acest mod de adresare este foarte important în procesoarele DSP la care operanzii pentru instrucțiunile aritmetice provin din registre și nu din memorie.

6.4 Adresarea directă la memorie

În cazul adresării directe la memorie (denumită și adresare absolută), datele care trebuie adresate se află în memorie la o adresă codată în cuvântul instrucțiunii.

La fel ca în cazul adresării imediate, adresele cu format mic pot fi codate în cuvântul instrucțiune, rezultând o instrucțiune de un singur cuvânt.

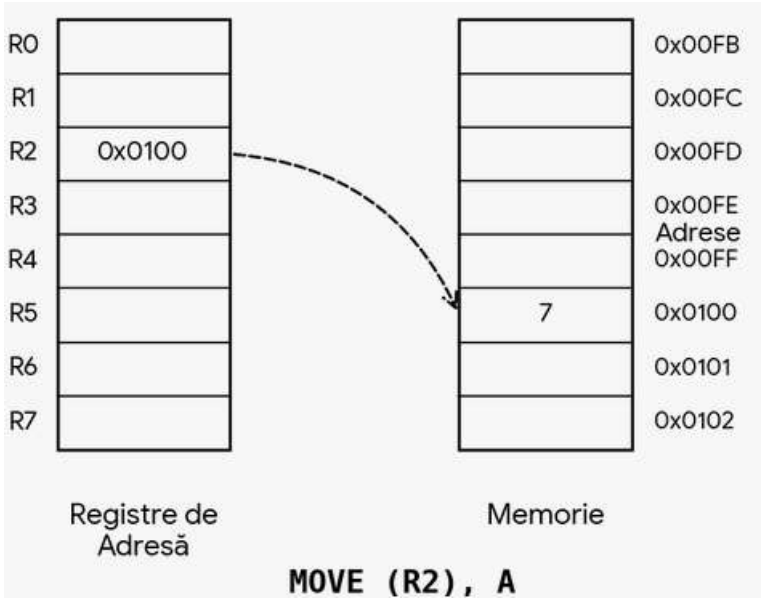
Adresele cu format mare trebuie plasate într-un cuvânt separat de cel al instrucțiunii. În acest caz procesorul trebuie să citească două cuvinte din memoria de program înainte ca instrucțiunea să poată fi executată, încetinind execuția programului.

6.5 Adresarea indirectă cu registru

Datele adresate se află în memorie la o adresă indicată de un registru.

Adresarea indirectă cu registru este cel mai folosit mod de adresare de pe majoritatea procesoarelor DSP.

Exemplu de adresare indirectă prin registru. Registrul R2 conține adresa 0x0100. Când este executată instrucțiunea de mutare (MOVE), valoarea stocată la locația de memorie 0x0100 (în acest caz 7) este copiată în acumulatorul A.



Un exemplu de adresare indirectă cu registru se poate considera instrucțiunea pentru AT&T: A0 = A0 + *R5, care are ca efect adunarea valorii stocate la locația de memorie indicată de conținutul registrului R5 la valoarea stocată în registrul acumulator A0. Rezultatul este depus în registrul acumulator A0. În această instrucțiune, operandul de intrare „*R5” este specificat folosind adresarea indirectă prin registru: R5 este folosit drept registru de adresă, acesta stochează adresa locației de memorie care conține operandul propriu-zis.

6.6 Adresarea indirectă cu registru cu pre- și post-incrementare/decrementare (1/2)

Cea mai comună modificare efectuată asupra unui registru de adrese constă în adăugarea unui increment cu valoarea 1 (unu) după ce registrul de adrese este folosit la accesarea operandului. De exemplu, instrucțiunea pentru procesorul DSP32xx:

A0 = A0 + *R5++

adună valoarea din locația de memorie indicată de R5 cu cea din acumulatorul A0 și apoi incrementează valoarea din registrul de adrese R5 astfel încât să specifice locația de memorie următoare.

În mod similar, instrucțiunea

A0 = A0 + *R5--

are ca efect decrementarea valorii registrului R5 pentru a indica locația de memorie anterioară după ce operația de adunare a fost efectuată.

6.6 Adresarea indirectă cu registru cu pre- și post-incrementare/decrementare (2/2)

Unele procesoare au posibilitatea de a adăuga sau scădea valori diferite de 1 (unu) registrului de adrese. De exemplu, instrucțiunea procesorului DSP32xx

A0 = A0 + *R5++R17

adună valoarea din locația de memorie indicată de registrul R5 cu cea din acumulator și apoi incrementează valoarea din registrul de adrese R5 cu valoarea stocată în registrul R17. Deoarece valoarea din R17 poate fi negativă, același format de instrucțiune poate fi folosit și pentru a decrementa valoarea stocată într-un registru de adrese.

De exemplu, la procesoarele Motorola, preincrementarea necesită de obicei un ciclu instrucțiune în plus pentru actualizarea registrului de adrese înaintea folosirii acestuia la adresarea memoriei.

6.7 Adresarea indirectă cu registru cu indexare

Valorile stocate în două registre pentru adresă (sau într-un registru pentru adresă și o constantă) sunt adunate pentru a rezulta adresa efectivă ce va fi utilizată la adresarea memoriei.

Diferența între adresarea indirectă cu indexare și adresarea indirectă cu premodificare constă în faptul că la indexare valorile stocate în registre nu sunt modificate.

Unii producători folosesc termenul de *adresare indexată* pentru a denumi adresarea indirectă cu registru cu pre- și post- incrementare cu o valoare diferită de unu. La aceste procesoare, registrele denumite anterior registre offset poartă denumirea de *registre index*.

La unele procesoare valoarea index poate fi furnizată ca o valoare imediată codată în instrucțiune.

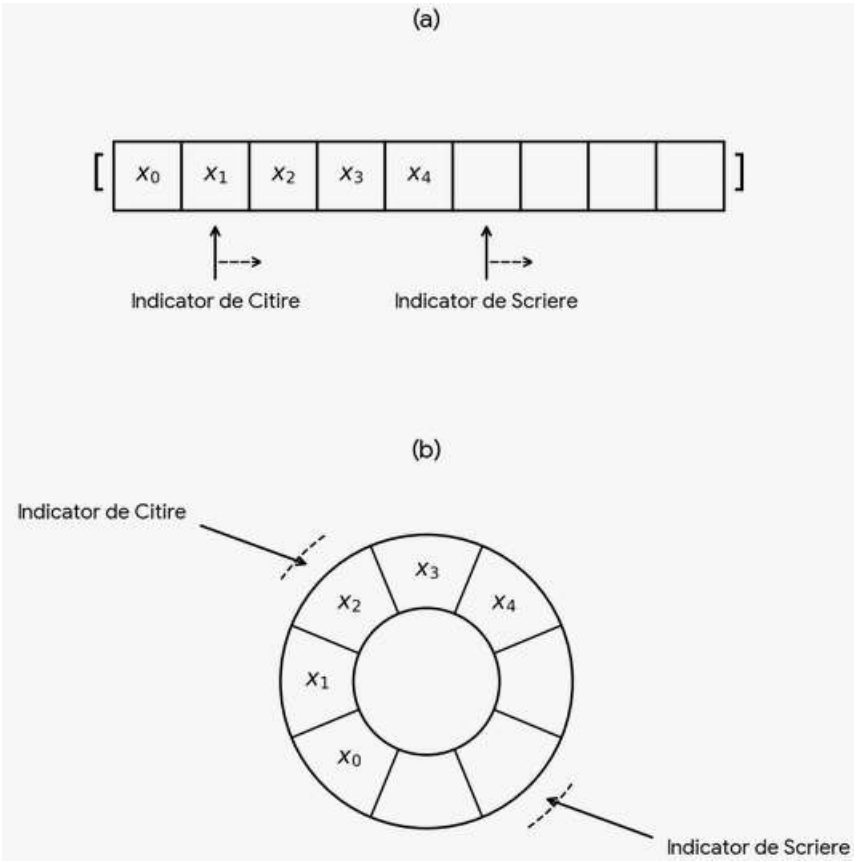
6.8 Adresarea indirectă cu registru cu aritmetică a adreselor modulo

Cea mai comună aplicație a acestui mod de adresare este implementarea unui buffer circular, mai ales că multe aplicații DSP necesită implementarea unor buffere.
Un buffer de date este o porțiune de memorie utilizată pentru a stoca datele provenite dintr-o sursă din exteriorul chipului sau de la o procesare anterioară până când procesorul este gata să le prelucreze.
La unele sisteme în timp real programatorul trebuie să determine cantitatea maximă de date pe care un buffer trebuie să o memoreze și să rezerve o porțiune de memorie pentru acesta (de obicei unde alocarea dinamică a memoriei implică un preț destul de mare). Aceste buffere folosesc tehnica FIFO care constă în citirea datelor din buffer în ordinea în care au sosit.
La implementarea mișcării datelor în buffer, programatorul folosește doi pointeri care sunt stocați în memorie sau în regiștri: un pointer de scriere și un pointer de citire. Pointerul de scriere indică locația de memorie unde va fi depusă următoarea valoare sosită, iar pointerul de citire indică locația de memorie unde se află valoarea următoare ce va fi citită. De fiecare dată când se efectuează o operație de scriere sau de citire, cei doi pointeri avansează, iar programatorul trebuie să verifice dacă s-a ajuns la ultima locație a buffer-ului. Testul dacă pointerul a ajuns la sfârșitul buffer-ului după fiecare operație și resetarea acestuia dacă a ajuns, consumă mult timp. La sistemele care utilizează astfel de buffere aceasta poate fi o cauză de scădere semnificativă a performanței.

6.8 Adresarea indirectă cu registru cu aritmetică a adreselor modulo

(a) Un tampon FIFO cu adresare liniară. Cinci elemente de date x_n au sosit în ordinea x_0, x_1, x_2, x_3, x_4 și au fost scrise în tampon. Doar primul element de date, x_0, a fost citit din tampon. După fiecare operație de citire sau scriere, indicatorul corespunzător se deplasează spre dreapta. Odată ce oricare dintre indicatori ajunge la sfârșitul tamponului, acesta trebuie resetat pentru a indica începutul tamponului.

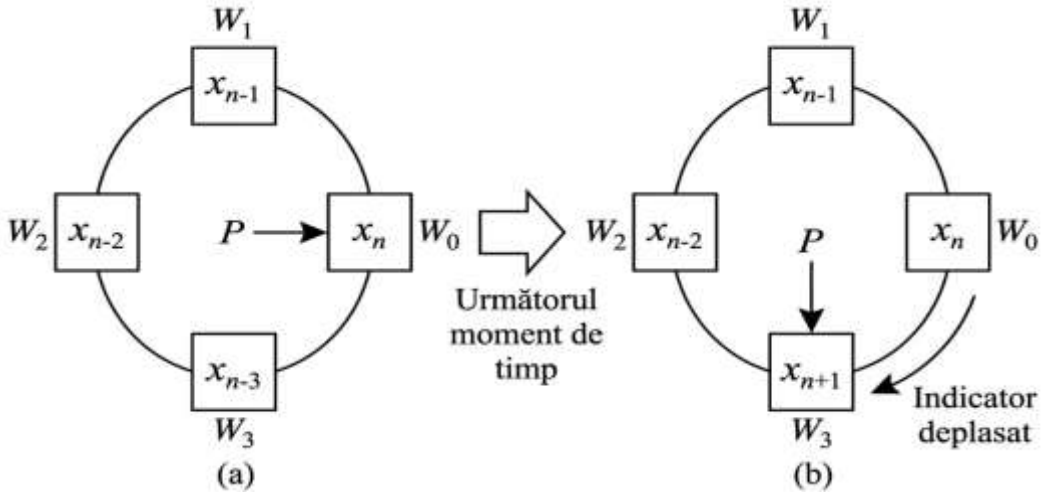
(b) Aceleași date într-un tampon FIFO cu adresare circulară. După ce indicatorul de citire sau cel de scriere ajunge la sfârșitul tamponului, acesta avansează automat la începutul tamponului, făcând ca tamponul să pară circular pentru programator.



6.8 Adresarea indirectă cu registru cu aritmetică a adreselor modulo

Pentru a înlătura acest dezavantaj, multe DSP-uri au un mod de adresare special care le permite să efectueze automat aceste verificări după fiecare operație de calcul a adreselor bufferului și să ajusteze pointerul la începutul bufferului dacă este necesar. Aceasta capabilitate se numește adresare modulo sau adresare circulară. Principiul de funcționare este următorul:

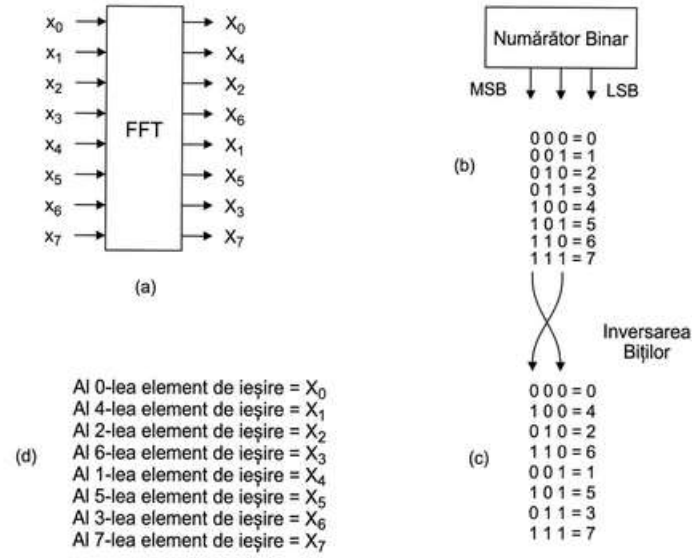
Funcționarea unui buffer circular utilizat pentru implementarea unui filtru FIR. (a) La momentul curent, indicatorul P indică locația unde este stocat cel mai recent eșantion, x_n. Acest eșantion este multiplicat cu coeficientul w_0, în timp ce eșantioanele anterioare sunt asociate cu coeficienții corespunzători. (b) La următorul moment de timp, noul eșantion sosit (x_{n+1}) rescrie cea mai veche valoare din buffer (x_{n-3}), iar indicatorul P este deplasat în mod circular pentru a indica această nouă poziție.



6.9 Adresarea indirectă cu registru cu inversarea biților

Unele aplicații DSP folosesc mult algoritmi de transformare Fourier rapidă (FFT). O categorie importantă de algoritmi FFT sunt algoritmii în baza 2. Există două categorii de algoritmi în baza 2: cu decimare în timp și cu decimare în frecvență. Această categorie de algoritmi prezintă însă dezavantajul ca ori folosesc date de intrare într-o ordine inversată, ori produc datele de ieșire într-o ordine inversată. Această situație impune rearanjarea datelor în ordinea firească la un moment dat.

Tipul de inversare depinde de variantele algoritmului FFT folosit. Implementarea FFT în baza 2, necesită reordonarea datelor într-un mod simplu denumit ordine inversă a biților. Termenul de inversare a biților se referă la observarea faptului că dacă valorile de ieșire ale unui numărător binar sunt scrise în ordine inversă (bitul cel mai puțin semnificativ este primul scris) secvența rezultată se va potrivi cu ieșirea inversată a datelor de ieșire a algoritmului FFT.



Ieșirea unui algoritm FFT (în acest caz, o FFT în baza 2 cu opt puncte) este produsă într-o ordine amestecată (a). Dacă ieșirea unui numărător binar (b) are biții inversați secvența rezultată (c) poate fi utilizată pentru a transforma ieșirea FFT în ordinea sa naturală (d) pentru procesare ulterioară.

6.10 Modurile de adresare scurte (1/3)

Un neajuns al câtorva moduri de adresare este acela că în general instrucțiunea și adresa necesită două cuvinte de program pentru memorare, ceea ce mărește dimensiunea programului și încetinește execuția.

Unele procesoare oferă versiuni scurte ale unor moduri de adresare. Aceste versiuni scurte includ într-un singur cuvânt atât instrucțiunea cât și adresa, dar pe seama unor restricții la nivelul adreselor.

a) Adresarea scurtă cu date imediate;

b) Adresarea scurtă directă la memorie.

6.10 Modurile de adresare scurte (2/3)

a) Adresarea scurtă cu date imediate

În modul de adresare cu date imediate operandul propriu-zis (și nu adresa unde este stocat) este codat în cuvântul instrucțiune sau într-un cuvânt de date ce urmează cuvântului instrucțiune.

În sintaxa limbajului de asamblare al procesoarelor Motorola, de exemplu, adresarea imediată poate fi folosită la încărcarea unei constante într-un registru, cum ar fi în cazul instrucțiunii

MOVE #1234, A

care încarcă valoarea constantă în registrul A.

În general, instrucțiunea necesită două cuvinte de memorie de program: unul pentru stocarea codului operației de încărcare și unul pentru stocarea datelor imediate. Totuși, dacă datele imediate sunt mici, ar fi posibilă includerea instrucțiunii și a datelor într-un singur cuvânt. În cazul Motorola, instrucțiunile cu date imediate în forma de mai sus pot fi codate într-un singur cuvânt de memorie de program dacă datele au 12 biți sau mai puțin.

6.10 Modurile de adresare scurte (3/3)

b) Adresarea scurtă directă la memorie

Din cazul prezentat anterior referitor la adresarea directă la memorie, datele ce urmează a fi adresate se află în memorie, și adresele la care acestea se află sunt codate în cuvântul instrucțiune sau într-un cuvânt care urmează instrucțiunii. De exemplu, la procesoarele Motorola, instrucțiunea în limbaj de asamblare

MOVE $1000, A

are ca efect încărcarea datelor aflate la adresa 1000 în registrul A.

În acest procesor produs de compania Motorola, adresarea scurtă directă la memorie permite codarea în instrucțiune a adreselor de până la șase biți lungime. Această soluție economisește un cuvânt de memorie (și un ciclu instrucțiune) comparativ cu varianta de adresare directă la memorie descrisă anterior.

În unele cazuri, procesoarele pot pune la dispoziție și alte versiuni de adresare scurtă directă la memorie care adaugă un offset implicit la adresa scurtă. De exemplu, procesorul folosește un mod de adresare scurtă directă la memorie pentru adresarea registrelor I/O care sunt mapate pe ultimele 64 locații de memorie.

6.11 Adresarea directă la memorie paginată

În acest caz, un registru special stochează numărul paginii (secțiuni) de memorie care este adresată. Astfel o anumită instrucțiune poate adresa direct un cuvânt în interiorul paginii specificând locația relativă a cuvântului în pagină. Pentru adresarea unui cuvânt din afara paginii, registrul de pagină trebuie actualizat (sau se poate utiliza un alt registru de pagină dacă sunt mai multe disponibile).

De exemplu, deși procesoarele de la Texas Instruments pot adresa până la 64 Kcuvinte de memorie de date, instrucțiunile care folosesc adresarea scurtă directă la memorie nu folosesc direct o adresă de 16 biți.

Instrucțiunile care folosesc acest mod de adresare au 7 biți de adrese, suficienți pentru a selecta un cuvânt din 128. Procesorul împarte memoria în pagini de 128 cuvinte fiecare. Programatorul selectează o pagină încărcând numărul paginii în registrul indicator de pagină. Instrucțiunile care urmează pot executa accesări în interiorul paginii de 128 cuvinte fără a mai fi nevoie de un al doilea cuvânt de memorie de program care să specifice adresa.

Această tehnică este eficientă doar dacă programatorul poate organiza programul astfel încât să aibă loc un număr semnificativ de accesări în interiorul unei pagini înainte de a fi nevoie de utilizarea altei pagini.